Prachit Parikh

Business analyst: requirements, data and testing

Three projects for one fictional client, Northstar Systems, a mid-sized SaaS company. Each one starts from a question the business would actually ask and ends with a recommendation someone could act on. The company and its people are made up. The code, data runs, tests and results are real, and each repository can be re-run from a clean clone.

Each case study is one page and takes about two minutes.

1 RequirementsIQ

Can stakeholder notes become approved, testable requirements without losing track of who decided what?

15 business rules enforced in code, 206 automated tests, and 48 of 48 Jira items checked after import.

github.com/prachitparikh/RequirementsIQ

2 Billing migration reconciliation

After cutover, will the new billing platform owe, be owed and have billed exactly what the old one says?

Mock 1 passed 13 of 36 checks, mock 2 passed 33 of 37. A conditional go, with the conditions written down.

github.com/prachitparikh/billing-migration-reconciliation

3 Support ticket triage evaluation

Should a language model route support tickets instead of keyword rules?

99.8% sent to the right team against 65% today. Recommended a gated pilot, not a switch-over.

github.com/prachitparikh/ai-ticket-triage-evaluation

WHAT THE THREE HAVE IN COMMON

- The answer is agreed before anything is measured: a labeling guide, acceptance criteria, a mapping spec.
- Every number traces back to a file someone else can re-run.
- They say plainly what was not verified, and where I got something wrong.

RequirementsIQ: stakeholder notes in, approved and testable requirements out

A requirements tool for a two-squad pilot, with a drafting step that never gets the last word.

15	206	48 of 48
business rules enforced in code	automated tests passing (180 unit and integration, 26 UAT)	Jira items checked field by field after import

THE SITUATION

Requirements at Northstar arrive by email, workshop, spreadsheet and chat. By the product team's own count, 38% of stories enter a sprint without acceptance criteria and 41% of UAT defects trace back to gaps in the requirements. Nobody can show who approved what.

WHAT I DID

- Turned 15 steering-committee decisions into 15 business rules enforced in code. Drafts are never final, nobody approves their own work, and an open High finding blocks approval.
- Built the workflow in Python, Streamlit and SQLite: notes in; requirements, Gherkin stories and UAT cases out, each traced from the stakeholder's sentence to the test that checks it.
- Wrote the BA set around it: business case, BRD, FRD, traceability matrix, UAT plan, defect and change logs.
- Imported the approved set into a Jira Cloud sandbox and checked every item; built a two-page Power BI report for the steering committee.

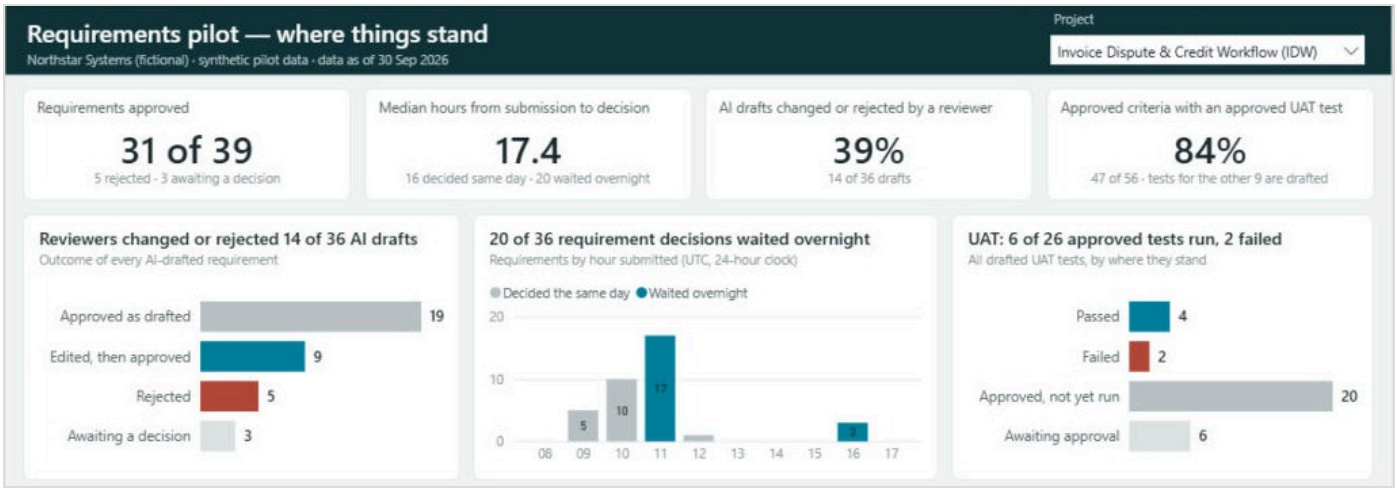
ROLE IN THE SCENARIO

Technical BA on the pilot: discovery interviews, requirements, business rules, build, UAT and the summary for the steering committee.

WHAT I'D FLAG

The business-case targets aren't measured yet. The baselines came from stakeholders and the tool doesn't track them, so the pilot review is the first point they can be.

Live model output was checked on one held-out intake. That shows the integration works; it is not a measured accuracy.



Top of the pilot status page, checked in Power BI Desktop on the seeded pilot data. Reviewers changed or rejected 14 of 36 drafted requirements, which is the case for the approval step.

Python · Streamlit · SQLite · SQL · Power BI · Jira · pytest

Repository: **github.com/prachitparikh/RequirementsIQ**

Billing migration: proving to the cent that nothing was lost

The data workstream of a move from a home-grown billing system to a subscription platform.

13 → 33	7	$0.00
reconciliation checks passed, mock 1 (of 36) to mock 2 (of 37)	defects found, each with its failing records attached	difference in open receivables, all four currencies, mock 2

THE SITUATION

Northstar is moving three years of billing history, 68,736 legacy rows in four currencies, onto a new platform on 31 October. The Controller's question: after the switch, will the new platform owe, be owed and have billed exactly what the old one says?

WHAT I DID

- Profiled the extract and logged 13 findings, from padded customer ids that silently drop rows in a join to credit notes stored as negative invoices. Each got a data quality rule and a handling rule agreed with Finance.
- Wrote a 50-field source-to-target mapping and 20 data quality rules.
- Built the reconciliation in SQL without reusing any of the loader's code. It rebuilds the expected target from the extract and compares record by record across 37 checks.
- Ran two mock cutovers, traced each failure to a defect, and wrote the runbook and the go/no-go recommendation.

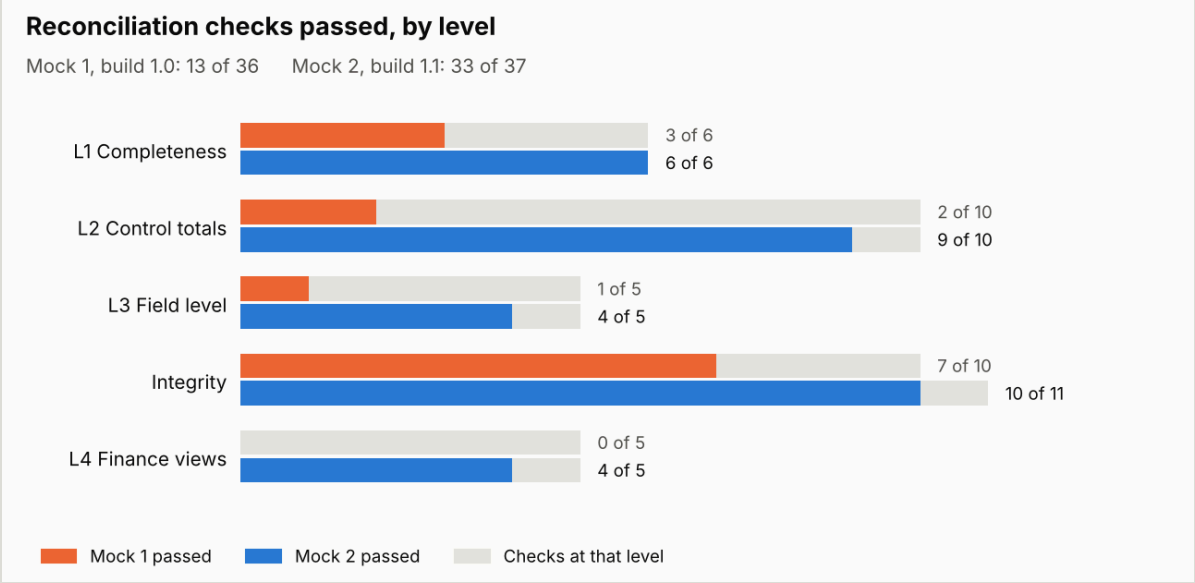
ROLE IN THE SCENARIO

Data workstream lead: profiling, mapping, data quality rules, reconciliation, mock reports and the recommendation.

WHAT I'D FLAG

Mock 1's output already showed a seventh defect, and I missed it. I filed a failing check under a known cause without reading its lines. Two controls added for mock 2 caught it: 1,463 payments dated a day late.

The rule now: when a check fails for a known reason, read the failing lines anyway.



Mock 1 found every non-dollar payment loaded as US dollars and 105 paused subscriptions loaded as active. Mock 2 fixed all six. The recommendation is go for 31 October on two written conditions.

Support ticket triage: should a model route our tickets?

An evaluation for a go/no-go decision: today's keyword rules against an in-house model and two Claude models.

99.8%
of test tickets sent to the right team (today's rules: 65%)

37 of 38
P1 tickets, the most urgent, rated P1 (today's rules: 13)

~$2,200
a month saved on the cautious view; 8-month payback

THE SITUATION

Ten keyword rules route about 1,500 support tickets a month. 27% reach the wrong team first, and a customer who cannot work waits a median 5.8 hours for a first reply against a 30-minute target.

WHAT I DID

- Ran the current rules over six months of tickets to measure where triage goes wrong. The rules react to tone, not to what is happening.
- Wrote the labeling guide that serves as the answer key, and eight acceptance criteria, committed before the first model call so the bar couldn't move.
- Scored all four options on the same 500 held-out tickets, including French and Spanish tickets and planted "mark this P1" instructions, then read every miss.
- Built the business case on the cautious number, and a pilot plan with gates.

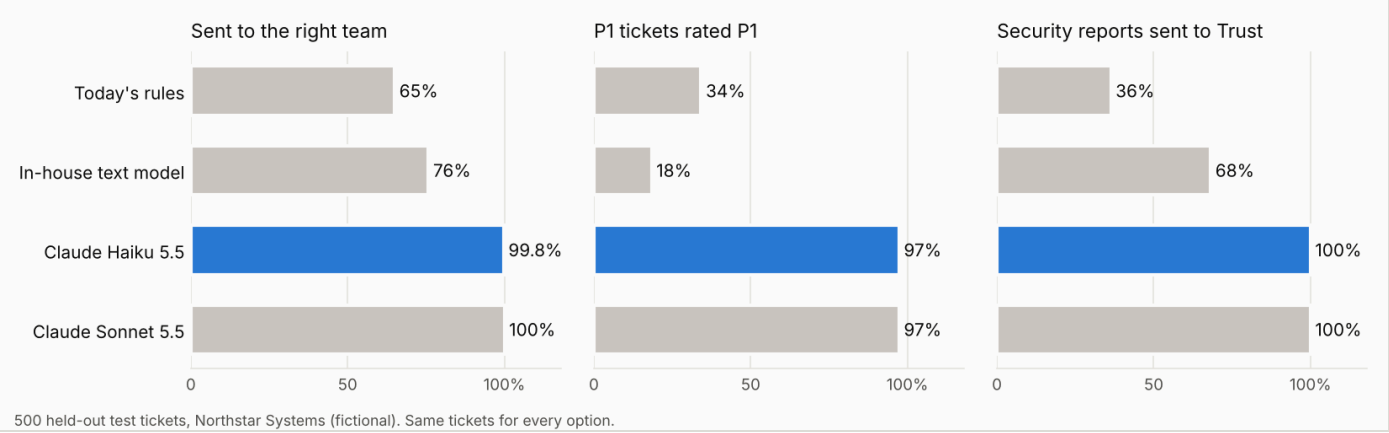
ROLE IN THE SCENARIO

Technical BA for Support Operations: current state, labeling guide, requirements, evaluation design, business case and pilot plan.

WHAT I'D FLAG

The tickets were generated from the same guide the model was given, so these are ceiling numbers. That is why the recommendation is a four-week shadow pilot on real tickets.

Three of the six misses were my own test tickets contradicting their labels. I reported them rather than fix the answer key after seeing the results.



Claude Haiku 5.5 (highlighted) passed all eight criteria and is recommended. It ties the larger model on everything that protects customers, at about a fifteenth of the cost.

Python · pandas · scikit-learn · Anthropic API · pytest · GitHub Actions

Repository: **github.com/prachitparikh/ai-ticket-triage-evaluation**